Thirteen Definitions of a Stable Model

Vladimir Lifschitz

Department of Computer Sciences, University of Texas at Austin, USA
vl@cs.utexas.edu

Abstract. Stable models of logic programs have been studied by many researchers, mainly because of their role in the foundations of answer set programming. This is a review of some of the definitions of the concept of a stable model that have been proposed in the literature. These definitions are equivalent to each other, at least when applied to traditional Prolog-style programs, but there are reasons why each of them is valuable and interesting. A new characterization of stable models can suggest an alternative picture of the intuitive meaning of logic programs; or it can lead to new algorithms for generating stable models; or it can work better than others when we turn to generalizations of the traditional syntax that are important from the perspective of answer set programming; or it can be more convenient for use in proofs; or it can be interesting simply because it demonstrates a relationship between seemingly unrelated ideas.

1 Introduction

Stable models of logic programs have been studied by many researchers, mainly because of their role in the foundations of answer set programming (ASP) [30, 37]. This programming paradigm provides a declarative approach to solving combinatorial search problems, and it has found applications in several areas of science and technology [21]. In ASP, a search problem is reduced to computing stable models, and programs for generating stable models ("answer set solvers") are used to perform search.

This paper is a review of some of the definitions, or characterizations, of the concept of a stable model that have been proposed in the literature. These definitions are equivalent to each other when applied to "traditional rules" – with an atom in the head and a list of atoms, some possibly preceded with the negation as failure symbol, in the body:

$$A_0 \leftarrow A_1, \ldots, A_m, not\ A_{m+1}, not\ A_n. \tag{1}$$

But there are reasons why each of them is valuable and interesting. A new characterization of stable models can suggest an alternative picture of the intuitive meaning of logic programs; or it can lead to new algorithms for generating stable models; or it can work better when we turn to generalizations of the traditional syntax that are important from the perspective of answer set programming; or it can be more convenient for use in proofs, such as proofs of correctness of ASP